

Bases de Datos II

Trabajo Práctico 5

Nombre: Farias, Gustavo

Comisión: M2025-13

Matrícula: 101662

Repositorio GitHub:

https://github.com/Lucenear/Tecnicatura_Programacion_UTN/tree/main/3er_Semestre/BBD_D_II

ACTIVIDAD 1: Optimización mediante Índices

- A. El campo numero_seguimiento requiere un Índice Simple (y único). Comando: db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })
- B. Los campos estado y ciudad_destino requieren un Índice Compuesto. Comando: db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
- C. Sin índices, MongoDB realiza un COLLSCAN (Collection Scan), leyendo cada documento de la colección secuencialmente. Esto consume mucha CPU y E/S, volviéndose extremadamente lento a medida que crece el volumen de datos.

Implementacion practica:

- Creación de base de datos

```
test> use logistica_db
switched to db logistica_db
logistica_db> █
```

- Creación de coleccion

```
logistica_db> db.envios.insertMany([
| { numero_seguimiento: "TRK001", estado: "En Transito", ciudad_destino: "Buenos Aires", peso: 5.2 },
| { numero_seguimiento: "TRK002", estado: "Entregado", ciudad_destino: "Cordoba", peso: 12.0 },
| { numero_seguimiento: "TRK003", estado: "Pendiente", ciudad_destino: "Rosario", peso: 2.5 },
| { numero_seguimiento: "TRK004", estado: "Entregado", ciudad_destino: "Buenos Aires", peso: 25.0 },
| { numero_seguimiento: "TRK005", estado: "En Transito", ciudad_destino: "Mendoza", peso: 8.7 },
| { numero_seguimiento: "TRK006", estado: "Entregado", ciudad_destino: "Cordoba", peso: 15.3 },
| { numero_seguimiento: "TRK007", estado: "Cancelado", ciudad_destino: "La Plata", peso: 1.0 },
| { numero_seguimiento: "TRK008", estado: "Entregado", ciudad_destino: "Buenos Aires", peso: 30.0 },
| { numero_seguimiento: "TRK009", estado: "Pendiente", ciudad_destino: "Mar del Plata", peso: 4.5 },
| { numero_seguimiento: "TRK010", estado: "Entregado", ciudad_destino: "Salta", peso: 18.2 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a33cf11b575be3e95597e0f'),
    '1': ObjectId('6a33cf11b575be3e95597e10'),
    '2': ObjectId('6a33cf11b575be3e95597e11'),
    '3': ObjectId('6a33cf11b575be3e95597e12'),
    '4': ObjectId('6a33cf11b575be3e95597e13'),
    '5': ObjectId('6a33cf11b575be3e95597e14'),
    '6': ObjectId('6a33cf11b575be3e95597e15'),
    '7': ObjectId('6a33cf11b575be3e95597e16'),
    '8': ObjectId('6a33cf11b575be3e95597e17'),
    '9': ObjectId('6a33cf11b575be3e95597e18')
  }
}
```

- Creacion de indices

```
logistica_db> db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })
|
| db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
estado_1_ciudad_destino_1
```

- Codigo completo

```
use logistica_db

db.envios.insertMany([
  { numero_seguimiento: "TRK001", estado: "En Transito", ciudad_destino:
"Buenos Aires", peso: 5.2 },
```

```

    { numero_seguimiento: "TRK002", estado: "Entregado", ciudad_destino:
"Cordoba", peso: 12.0 },
    { numero_seguimiento: "TRK003", estado: "Pendiente", ciudad_destino:
"Rosario", peso: 2.5 },
    { numero_seguimiento: "TRK004", estado: "Entregado", ciudad_destino:
"Buenos Aires", peso: 25.0 },
    { numero_seguimiento: "TRK005", estado: "En Transito", ciudad_destino:
"Mendoza", peso: 8.7 },
    { numero_seguimiento: "TRK006", estado: "Entregado", ciudad_destino:
"Cordoba", peso: 15.3 },
    { numero_seguimiento: "TRK007", estado: "Cancelado", ciudad_destino:
"La Plata", peso: 1.0 },
    { numero_seguimiento: "TRK008", estado: "Entregado", ciudad_destino:
"Buenos Aires", peso: 30.0 },
    { numero_seguimiento: "TRK009", estado: "Pendiente", ciudad_destino:
"Mar del Plata", peso: 4.5 },
    { numero_seguimiento: "TRK010", estado: "Entregado", ciudad_destino:
"Salta", peso: 18.2 }
])

db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })

db.envios.createIndex({ estado: 1, ciudad_destino: 1 })

```

ACTIVIDAD 2: Diagnóstico y Evaluación de Rendimiento

a) El comando es `.explain("executionStats")`

b) `executionTimeMillis`: Tiempo total en milisegundos que tomó ejecutar la consulta.

`totalDocsExamined`: Cantidad de documentos que MongoDB tuvo que leer/descartar para encontrar los resultados.

c) La solución recomendada es implementar Proyecciones (usando `.find({}, {campo: 1})`) para traer solo los campos necesarios, o usar el operador `$limit` si no se necesitan todos los documentos, reduciendo así el uso de memoria.

Implementacion practica:

```

logistica_db> db.envios.find({ numero_seguimiento: "TRK001" }).explain("executionStats")
{
  db.envios.stats()
  {
    ok: 1,
    capped: false,
    wiredTiger: {
      metadata: { formatVersion: 1 },
      creationString: "index:splitter.btree:allocation_size=40,app.metadata=formatVersion=1,assert=commit.timestamp:none,durable.timestamp:none,read.timestamp:none,write.timestamp:none,block_allocation=best,block_compressor=snappy,cache.resident=false,checksums=colgroups,collator=,columns=,dictionary=0,encryption=keyids,exclusive=false,extractor=,format=btree,huffman_keys,huffman_values,ignore_in_memory_cache_size=false,immutable=false,import=compare.timestamp:disabled,timestamp.enabled=false,file.metadata=metadata,file.panic.corrupt=true,repair=false,internal.item_max=,internal.key_max=,internal.key.truncate=true,internal.page_max=4096,key.format=,key.page=10,leaf.item_max=,leaf.key_max=,leaf.page_max=1024,leaf.value_max=4096,log.enabled=true,log.entry.throttle=true,bloom=true,bloom.bit_count=1,bloom.config=bloom_hash,count=,bloom_offset=false,chunk.count.limit=,chunk.max=500,chunk.size=100,merge.cost=,merge.prefix=,start_generation=0,suffix=,merge_max=15,merge_min=0,memory.page_usage_max=10,os.cache_dirty_max=0,os.cache_max=,prefix_compression=false,prefix_compression_min=4,source=,split_dampen_min_child=,split_dampen_per_child=,split_pct=90,tiered_storage=,auth.token=,bucket.prefix=,cache_directory=,local.retention=300,name=,object.target_size=0,type=file,value.format=u,verbose=,write.timestamp_usage=none",
      type: file,
      url: "statistics:table/collection-2-1438174225578687272",
      LSM: {
        bloom filter false positives: 0,
        bloom filter hits: 0,
        bloom filter misses: 0,
        bloom filter pages evicted from cache: 0,
        bloom filter pages read into cache: 0,
        bloom filters in the LSM tree: 0,
        chunks in the LSM tree: 0,
        highest merge generation in the LSM tree: 0,
        queries that could have benefited from a bloom filter that did not exist: 0,
        sleep for LSM checkpoint throttle: 0,
        sleep for LSM merge throttle: 0,
        total size of bloom filters: 0
      }
    }
  }
}

```

- Código

```
db.envios.find({ numero_seguimiento: "TRK001"
}).explain("executionStats")

db.envios.stats()
```

ACTIVIDAD 3: Procesamiento Avanzado con Agregaciones

a) El Pipeline de Agregación procesa documentos en etapas secuenciales. Funciona como una "tubería" porque la salida de una etapa es la entrada de la siguiente.

b) \$match: Filtra documentos (como un WHERE). \$group: Agrupa documentos por un criterio y permite calcular totales/promedios. \$project: Remodela los documentos, añadiendo, eliminando o renombrando campos.

c) Se utiliza el operador \$lookup (equivalente a un JOIN en SQL).

Implementación práctica

- Contar entregas por ciudad

```
logistica_db> db.envios.aggregate([
|   { $match: { estado: "Entregado" } },
|   { $group: { _id: "$ciudad_destino", total_entregas: { $sum: 1 } } }
| ])
[
|   { _id: 'Buenos Aires', total_entregas: 2 },
|   { _id: 'Cordoba', total_entregas: 2 },
|   { _id: 'Salta', total_entregas: 1 }
| ]
```

- Calcular peso total

```
logistica_db> db.envios.aggregate([
|   { $group: { _id: null, peso_total: { $sum: "$peso" } } }
| ])
[
|   { _id: null, peso_total: 122.4 } ]
```

- Numero de seguimiento en mayúsculas y estado

```
logistica_db> db.envios.aggregate([
|   { $project: { _id: 0, numero_seguimiento: { $toUpper: "$numero_seguimiento" }, estado: 1 } }
| ])
[
|   { estado: 'En Transito', numero_seguimiento: 'TRK001' },
|   { estado: 'Entregado', numero_seguimiento: 'TRK002' },
|   { estado: 'Pendiente', numero_seguimiento: 'TRK003' },
|   { estado: 'Entregado', numero_seguimiento: 'TRK004' },
|   { estado: 'En Transito', numero_seguimiento: 'TRK005' },
|   { estado: 'Entregado', numero_seguimiento: 'TRK006' },
|   { estado: 'Cancelado', numero_seguimiento: 'TRK007' },
|   { estado: 'Entregado', numero_seguimiento: 'TRK008' },
|   { estado: 'Pendiente', numero_seguimiento: 'TRK009' },
|   { estado: 'Entregado', numero_seguimiento: 'TRK010' }
| ]
```

- Campo nuevo tipo_carga

```
logistica_db> db.envios.aggregate([
|   { $addFields: { tipo_carga: { $cond: { if: { $gt: [ "$peso", 20 ] }, then: "Pesada", else: "Ligera" } } } }
| ])
[
|   {
|     _id: ObjectId('6a33cf11b575be3e95597e0f'),
|     numero_seguimiento: 'TRK001',
|     estado: 'En Transito',
|     ciudad_destino: 'Buenos Aires',
|     peso: 5.2,
|     tipo_carga: 'Ligera'
|   },
|   {
|     _id: ObjectId('6a33cf11b575be3e95597e10'),
|     numero_seguimiento: 'TRK002',
|     estado: 'Entregado',
|     ciudad_destino: 'Cordoba',
|     peso: 12,
|     tipo_carga: 'Ligera'
|   },
|   {
|     _id: ObjectId('6a33cf11b575be3e95597e11'),
|     numero_seguimiento: 'TRK003',
|     estado: 'Pendiente',
|     ciudad_destino: 'Rosario',
|     peso: 2.5,
|     tipo_carga: 'Ligera'
|   },
|   {
|     _id: ObjectId('6a33cf11b575be3e95597e12'),
|     numero_seguimiento: 'TRK004',
|     estado: 'Entregado',
|   }
| ]
```

- Codigo completo

```
// a) Contar entregas por ciudad
db.envios.aggregate([
  { $match: { estado: "Entregado" } },
  { $group: { _id: "$ciudad_destino", total_entregas: { $sum: 1 } } }
])

// b) Calcular peso total
db.envios.aggregate([
  { $group: { _id: null, peso_total: { $sum: "$peso" } } }
])

// c) Numero de seguimiento en mayúsculas y estado
db.envios.aggregate([
  { $project: { _id: 0, numero_seguimiento: { $toUpper:
"$numero_seguimiento" }, estado: 1 } }
])

// d) Campo nuevo tipo_carga
db.envios.aggregate([
  { $addFields: { tipo_carga: { $cond: { if: { $gt: [ "$peso", 20 ] },
then: "Pesada", else: "Ligera" } } } }
])
```